Step 1: semantic search, by hand

Goal. A customer types *"a couch my guests can sleep on"* in the Nordhjem search box and gets sofas that fold out into a bed, even though no product text contains the word "couch". You write this search yourself, in `api/search.py`, with the embedding model and NumPy only, so every step is visible. When it passes the check, the live page switches from "keyword placeholder" to "your search".

What is already done for you (plumbing): the 500 products are in Firestore, each with its vector of 768 numbers. Your job is everything that happens when a customer asks.

Contents

1 The problem with keyword search 1

2 What an embedding is 2

3 Measuring "points the same way": cosine similarity 2

3.1 Ingredient 1: the dot product . . . 2

3.2 Ingredient 2: the length (L2 norm) . . . 3

3.3 Putting it together: the cosine . . . 3

3.4 The shortcut: normalise once, then only dot products . . . 4

4 Two kinds of text: queries and documents 4

5 768 numbers, and the length trap 5

6 Filters: a mask 5

7 Putting it together: what `search()` does 6

8 How to know if the search is good 6

9 The exercise 7

10 Experiments, after it passes 7

1 The problem with keyword search

The page runs a keyword search today (`api/keyword_search.py`). It counts how many words of the question appear in a product's text. These are real results from it:

- *"a couch my guests can sleep on"*: 2 sofas and 3 rugs, all with the same score 0.43. Why 0.43: the question has 7 words, 3 of them are found, and $3/7 \approx 0.43$. The 3 words found are small common ones, not "couch".
- *"couch"*: nothing at all. No product text contains that word; the catalogue says "sofa".

The catalogue does have the right products, for example *"2-seat sofa in grey fabric. Folds out into a bed for guests"*. Keyword search matches **letters**. We want to match **meaning**. The reference solution of this step returns 5 such sofa beds for the same question, with scores around 0.71.

2 What an embedding is

An **embedding model** reads a text and returns a list of numbers, called a **vector**. Here the model is `gemini-embedding-001` on Vertex AI, and we ask for 768 numbers per text. The model was trained so that texts with similar meaning get vectors that point in a similar direction, in any language.

A worked example, with made-up vectors of only 3 numbers so you can follow by hand:

	Text	Vector
A	"sofa that folds out into a bed"	$[0.9,\ 0.1,\ 0.4]$
B	"a couch my guests can sleep on"	$[0.8,\ 0.2,\ 0.5]$
C	"wool rug, 160x230 cm"	$[0.1,\ 0.9,\ 0.1]$

A and B point almost the same way; C points elsewhere. So the search question becomes: *which product vectors point most like the question's vector?*

3 Measuring "points the same way": cosine similarity

We use the three vectors from Section 2:

$$A = [0.9,\ 0.1,\ 0.4] \qquad B = [0.8,\ 0.2,\ 0.5] \qquad C = [0.1,\ 0.9,\ 0.1]$$

The **cosine similarity** of two vectors a and b is

$$\cos(a, b) \ = \ \frac{a \cdot b}{\|a\| \times \|b\|}$$

It has two ingredients: the dot product $a \cdot b$ on top, and the lengths $\|a\|$ and $\|b\|$ below. We compute each one, step by step. Results are rounded to 3 decimals.

3.1 Ingredient 1: the dot product

Definition. Multiply the first numbers of both vectors, then the second numbers, then the third, and add the products:

$$a \cdot b = a_1 b_1 + a_2 b_2 + a_3 b_3$$

A and B.

$$A \cdot B = 0.9 \times 0.8 \ + \ 0.1 \times 0.2 \ + \ 0.4 \times 0.5 = 0.72 + 0.02 + 0.20 = 0.94$$

A and C.

$$A \cdot C = 0.9 \times 0.1 \;+\; 0.1 \times 0.9 \;+\; 0.4 \times 0.1 = 0.09 + 0.09 + 0.04 = 0.22$$

B and C.

$$B \cdot C = 0.8 \times 0.1 \;+\; 0.2 \times 0.9 \;+\; 0.5 \times 0.1 = 0.08 + 0.18 + 0.05 = 0.31$$

3.2 Ingredient 2: the length (L2 norm)

Definition. Square each number of the vector, add the squares, and take the square root:

$$\|a\| = \sqrt{a_1^2 + a_2^2 + a_3^2}$$

This is the **Euclidean norm**, also called the **L2 norm** (the "2" is the power in the squares). It is the straight-line length of the arrow. In 2 dimensions it is Pythagoras: the vector $[3, 4]$ has length $\sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$. In NumPy, `np.linalg.norm(v)` computes it.

Length of A.

$$\|A\| = \sqrt{0.9^2 + 0.1^2 + 0.4^2} = \sqrt{0.81 + 0.01 + 0.16} = \sqrt{0.98} \approx 0.990$$

Length of B.

$$\|B\| = \sqrt{0.8^2 + 0.2^2 + 0.5^2} = \sqrt{0.64 + 0.04 + 0.25} = \sqrt{0.93} \approx 0.964$$

Length of C.

$$\|C\| = \sqrt{0.1^2 + 0.9^2 + 0.1^2} = \sqrt{0.01 + 0.81 + 0.01} = \sqrt{0.83} \approx 0.911$$

3.3 Putting it together: the cosine

A and B (the sofa bed and the couch for guests):

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{0.94}{0.990 \times 0.964} = \frac{0.94}{0.955} \approx \mathbf{0.985}$$

A and C (the sofa bed and the rug):

$$\cos(A, C) = \frac{0.22}{0.990 \times 0.911} = \frac{0.22}{0.902} \approx \mathbf{0.244}$$

B and C (the couch for guests and the rug):

$$\cos(B, C) = \frac{0.31}{0.964 \times 0.911} = \frac{0.31}{0.879} \approx \mathbf{0.353}$$

Reading the result. The question is B. Its scores are 0.985 with the sofa bed A and 0.353 with the rug C. Ranked from high to low: A first, C second. That ranking is the search result.

The cosine is always between -1 and 1: 1 means exactly the same direction, 0 unrelated (a right angle), -1 opposite.

3.4 The shortcut: normalise once, then only dot products

Normalising means dividing every number of a vector by the vector's length. The result points the same way and has length exactly 1. We write it with a hat: $\hat{A}$.

Normalising A. The length of A is 0.990 (more exactly 0.98995), so

$$\hat{A} = \left[\frac{0.9}{0.98995}, \frac{0.1}{0.98995}, \frac{0.4}{0.98995}\right] \approx [0.909,\ 0.101,\ 0.404]$$

Check the new length: $\sqrt{0.909^2 + 0.101^2 + 0.404^2} = \sqrt{0.826 + 0.010 + 0.163} = \sqrt{0.999} \approx 1$ (not exactly 1 only because of rounding). In the same way, $\hat{B} \approx [0.830,\ 0.207,\ 0.518]$ and $\hat{C} \approx [0.110,\ 0.988,\ 0.110]$.

Why this helps. For vectors of length 1 the bottom of the cosine formula is $1 \times 1 = 1$, so the cosine *is* the dot product. Check with A and B:

$$\hat{A} \cdot \hat{B} = 0.909 \times 0.830 + 0.101 \times 0.207 + 0.404 \times 0.518 = 0.754 + 0.021 + 0.209 = 0.984$$

which is $\cos(A, B) \approx 0.985$ again (the last digit differs only because the three products were rounded).

All products at once: one matrix product. Put the normalised product vectors as the rows of a matrix (here 2 products, A and C; in the exercise 500), and multiply by the normalised question vector $\hat{B}$. Each row gives one dot product, that is one score:

$$\begin{bmatrix} 0.909 & 0.101 & 0.404 \\ 0.110 & 0.988 & 0.110 \end{bmatrix} \begin{bmatrix} 0.830 \\ 0.207 \\ 0.518 \end{bmatrix} = \begin{bmatrix} 0.909 \times 0.830 + 0.101 \times 0.207 + 0.404 \times 0.518 \\ 0.110 \times 0.830 + 0.988 \times 0.207 + 0.110 \times 0.518 \end{bmatrix} = \begin{bmatrix} 0.984 \\ 0.353 \end{bmatrix}$$

In NumPy this is one line; `@` is matrix multiplication:

```
scores = index.vectors @ question   # (500, 768) @ (768,) -> (500,)
```

4 Two kinds of text: queries and documents

Questions are short ("pillow for neck pain"); product texts are long and descriptive. The model can embed each **for its role**, set with `task_type`: the loader embedded the products as `"RETRIEVAL_DOCUMENT"`; you embed the question as `"RETRIEVAL_QUERY"`. The two kinds of vectors are made to match each other well.

What the loader embedded for each product is stored in Firestore as `embed_text`: name, category, colour and description. For product P1001:

> KILGAARD mattress. Category: mattress. Colour: grey. Memory foam mattress with medium firmness, 180x200 cm. Cooling cover that keeps you from getting too warm at night.

The vector only knows what is in this text. A product with a very short description still gets a useful vector because the name and category are added.

5 768 numbers, and the length trap

`gemini-embedding-001` returns 3072 numbers by default, already of length 1. It is trained so that the *first* numbers carry the most meaning (**Matryoshka** training, after the Russian dolls that fit inside each other), so we can ask for only the first 768: 4 times less storage, faster search, a small loss in quality. We use 768 because Firestore's own vector search (Step 3) accepts at most 2048 numbers.

The trap: shortened vectors are not length 1. Measured on this project:

- the product name "KILGAARD mattress", embedded as a document, 768 numbers: length 0.590;
- the question "a couch my guests can sleep on", 768 numbers: length 0.5905.

The loader already normalised the product vectors before storing them. **The question vector is yours to normalise.**

What happens if you forget. Write q for the question vector and $\hat{q} = q/0.5905$ for its normalised form, so $q = 0.5905 \times \hat{q}$. For any product $\hat{p}$:

$$\hat{p} \cdot q = \hat{p} \cdot (0.5905 \times \hat{q}) = 0.5905 \times (\hat{p} \cdot \hat{q}) = 0.5905 \times \cos(p, q)$$

Every score is multiplied by the same number 0.5905. Measured: the best sofa bed scores 0.716 as a cosine and 0.423 without normalising; check: $0.716 \times 0.5905 = 0.423$. Multiplying every score by the same positive number does *not* change their order, so the top 5 is the same. That is why the bug hides. It breaks anything that reads the score itself: a rule like "only show results above 0.6" would now show nothing, and scores of different questions can no longer be compared.

6 Filters: a mask

The page has a category choice and a max price. A price limit is a number to compare, not a meaning, so it is not done with vectors. It is done with a **mask**: one True or False per product, "may this product be shown?".

Worked example. Four products, max price 4000 kr:

Row	Score	Price (kr)	price ≤ 4000?	Score after the mask
0	0.82	3000	True	0.82
1	0.91	6500	False	$-\infty$
2	0.40	1200	True	0.40
3	0.77	2500	True	0.77

Row 1 has the best score but is too expensive. Setting its score to $-\infty$ (minus infinity, lower than any real score) pushes it to the very end. The top 2 is now row 0 (0.82), then row 3 (0.77).

In NumPy:

```
mask = index.prices <= 4000                 # [True, False, True, True]
scores = np.where(mask, scores, -np.inf)    # keep the score where True
```

Two conditions together use `&` ("and"), each in brackets:

```
mask = (index.categories == "sofa") & (index.prices <= 4000)
```

Two traps:

- If only 2 products pass the filters and the page asks for 24, return 2, not 24. Otherwise the last 22 are products with score $-\infty$, that is products that should not be shown.
- Real effect, measured with the reference: for the question "sofa under 4000 kr" with no filter, all 5 results cost between 6 569 and 10 269 kr. The words "under 4000" do not make a vector cheaper. With `max_price=4000` the mask fixes it. Reading the limit out of the question automatically is Step 3.

7 Putting it together: what search() does

`search(query, filters, k)` in `api/search.py`, step by step:

1. `load_index()`: read the 500 products from Firestore once and keep four arrays in the same order: IDs, vectors (500×768), categories and prices. Row i of each is product i. `@lru_cache` keeps the result, so later searches do not read Firestore again.
2. `embed_query(query)`: 768 numbers, as `RETRIEVAL_QUERY`.
3. `normalise(...)`: the question vector to length 1 (Section 5).
4. One matrix product: 500 scores (Section 3.4).
5. `allowed(index, filters)`: the mask; scores of the others become $-\infty$ (Section 6).
6. `top_k(scores, k)`: the row numbers of the k highest scores, best first; but never more than the number of allowed products.
7. Return `Hit(product_id=..., score=...)` for each row.

The API (`api/app.py`) turns the hits into product cards with name, price and photo. You do not touch it.

8 How to know if the search is good

Each of the 23 test questions in `data/eval/product_queries.json` lists the products that are correct for it. Two measures on the top 5:

- **hit@5**: is at least one correct product in the top 5? 1 or 0.
- **precision@5**: what share of the top 5 is correct? From 0 to 1.

Worked example (real, with the reference): "rug I can put in the washing machine". Of the 5 results, the first 4 are correct and the 5th is not. So hit@5 $= 1$ (at least one is correct) and precision@5 $= 4/5 = 0.8$. The check averages both over all questions of the same kind.

The questions have three kinds: `meaning` (15), `language` (5, in Danish or German) and `needs_filter` (3: "sofa under 4000 kr", "sofa that is not leather", "mattress 180x200"). The last kind is not graded: embeddings alone cannot do "under", "not" or exact sizes. The reference reaches hit@5 1.00 and precision@5 0.99 on `meaning`, 1.00 and 0.96 on `language`. To pass, both kinds together need hit@5 ≥ 0.95 and precision@5 ≥ 0.90.

9 The exercise

Exercise · fill in api/search.py

Fill in TODO 1 to TODO 6 in `api/search.py`, from top to bottom. After each function, run from the repo root:

```
uv run python steps/01_embeddings/check.py
```

Part 1 of the check tests `normalise`, `allowed` and `top_k` on the numbers of this document, without calling any service. Part 2 asks all 23 test questions.

What Part 1 checks, so you can predict it by hand:

- `normalise([0.9, 0.1, 0.4])` is about `[0.909, 0.101, 0.404]` (Section 3.4).
- `normalise` on the matrix with rows $[3, 4]$ and $[0, 2]$ gives rows $[0.6, 0.8]$ and $[0, 1]$. First row: length 5, then $3/5 = 0.6$ and $4/5 = 0.8$. Second row: length 2, then $0/2 = 0$ and $2/2 = 1$.
- `allowed` on the 4 products of Section 6 (categories sofa, sofa, rug, sofa): max price 4000 keeps rows 0, 2, 3; sofa and max price 4000 keeps rows 0 and 3; no filters keeps all four.
- `top_k([0.2, 0.9, 0.1, 0.7, 0.5], 3)` is `[1, 3, 4]`: the scores 0.9, 0.7 and 0.5 sit at positions 1, 3 and 4.

See it on the page. Run the API and the page locally (two terminals):

```
uv run uvicorn api.app:app --reload --port 8000
cd web && npm run dev
```

Search "a couch my guests can sleep on". The label at the right shows "your search" instead of "keyword placeholder". Try the max-price filter with "sofa under 4000 kr".

Live site. The deployed API runs as the service account `nordhjem-api`, which has no permissions yet. Before your search can run on `nordhjemstore.web.app`, that account needs to read Firestore and call Vertex AI (roles `datastore.user` and `aiplatform.user`). We do that together when you deploy.

10 Experiments, after it passes

Change one thing at a time, run the check again, and write the numbers in a comment at the top of `api/search.py`.

1. **Forget to normalise** the question on purpose. Do hit@5 and precision@5 change? Do the scores? Explain with Section 5.
2. **Task type.** Embed the question as `RETRIEVAL_DOCUMENT`. What happens to the scores, and to the ranking?
3. **Colour variants.** "a couch my guests can sleep on" returns the same HALDAGAARD sofa in grey, beige and dark green. How would you show 5 *different* products? (Hint: group by name, keep the best score.)

4. **The limits.** Look at what "sofa that is not leather" returns. Why can a vector not understand "not"? Write your explanation; we discuss it before Step 3.

The reference solution is `steps/01_embeddings/solution.py`. Open it only after trying, or when stuck on one function. `check.py --solution` runs the checks on it.